

## Módulo: Acceso a datos

# UT2: MANEJO DE CONECTORES

Código **0486**

Nivel **Ciclo Formativo de Grado Superior**
Ciclo **Desarrollo de Aplicaciones Multiplataforma**

# Índice

1 Introducción ....4
1.1 Resultados de aprendizaje y criterios de evaluación de la UT2 ....5
1.2 Conceptos previos ....5
2 Desfase objeto-relacional ....9
2.1 Origen del desfase objeto-relacional ....9
2.2 Soluciones al desfase objeto-relacional ....10
3 Protocolos de acceso a bases de datos: conectores ....11
3.1 Ejemplos más utilizados de conectores ....11
3.1.1 ODBC ....12
3.1.2 JDBC ....12
3.2 Conector JDBC ....12
3.2.1 Arquitectura de conexión JDBC ....12
3.2.2 Componentes JDBC ....13
3.2.3 Tipos de conectores JDBC ....14
3.3 Ventajas e inconvenientes de los conectores ....16
3.4 Elección del conector idóneo ....17
4 Establecimiento de conexiones ....18
4.1 Preparación de la base de datos: MySQL con Docker ....18
4.2 Driver de conexión y proyecto Maven ....20
4.3 Apertura de conexiones ....21
4.4 Gestores embebidos e independientes ....23
4.5 Realización de consultas ....24
5 Operaciones básicas ....26
5.1 La interfaz Statement ....26
5.2 Ejecución de sentencias de descripción de datos (DDL) ....27
5.3 Ejecución de consultas (DML - consultas) ....29
5.3.1 executeQuery ....29
5.3.2 Sentencias preparadas ....30
5.4 Ejecución de sentencias DML de modificación de datos ....32
6 Utilización del resultado de una consulta ....34
6.1 Tipos de ResultSet y su configuración ....34
6.2 Modificación de datos con ResultSet actualizable ....35

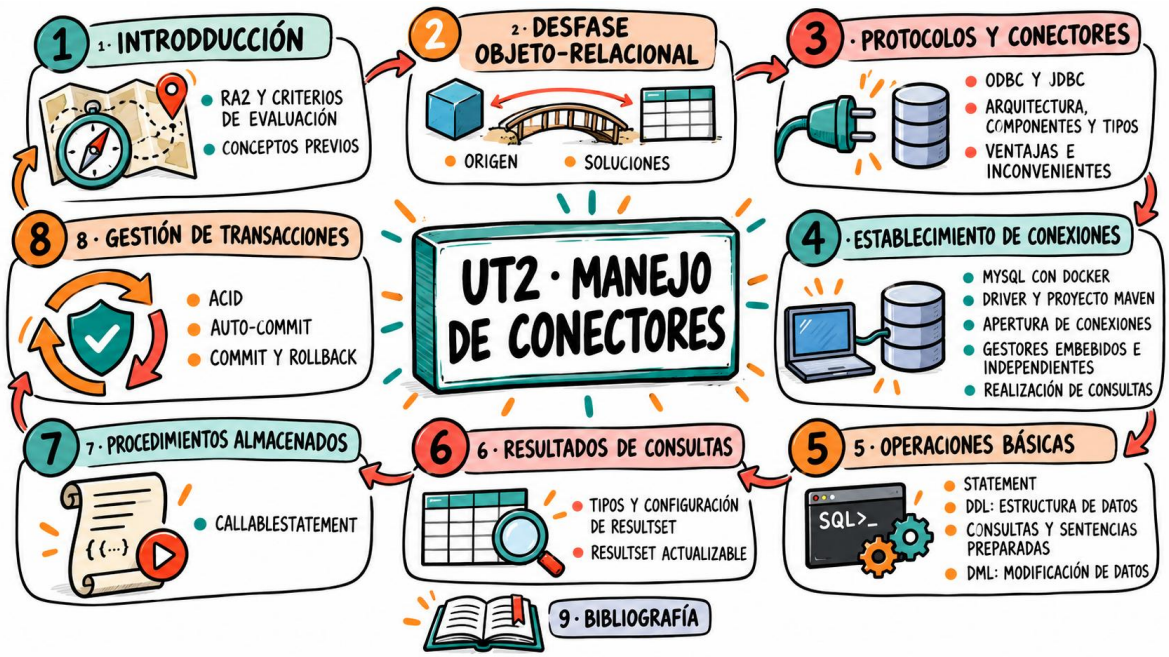
7 Ejecución de procedimientos almacenados en una base de datos .....38
7.1 La interfaz CallableStatement .....38
8 Gestión de transacciones .....42
Características ACID de las transacciones .....42
Implementación de transacciones en JDBC .....42
Pasos para gestionar una transacción en JDBC .....42
Ejemplo de transacción en JDBC .....43
Importancia de las transacciones .....44
9 Bibliografía .....44

# 1 Introducción

En la unidad de trabajo anterior aprendimos a guardar y recuperar información en ficheros. Los ficheros son una buena solución cuando el volumen de datos es pequeño y los usa una sola aplicación, pero se quedan cortos en cuanto la información crece, está relacionada entre sí (clientes, pedidos, productos…) y tienen que consultarla y modificarla varios usuarios a la vez. En esos casos, lo habitual es almacenar los datos en una **base de datos relacional**.

En esta unidad de trabajo aprenderemos a gestionar desde nuestras aplicaciones Java la información almacenada en bases de datos relacionales utilizando **conectores**. Empezaremos viendo por qué no es inmediato comunicar un programa orientado a objetos con una base de datos relacional (el desfase objeto-relacional) y qué son los conectores. Después aprenderemos a conectarnos tanto a un gestor independiente (**MySQL**) como a uno embebido (**SQLite**) y a ejecutar sobre ellos todo tipo de sentencias SQL: crear tablas, consultar, insertar, modificar y borrar datos, llamar a procedimientos almacenados y agrupar varias operaciones en una transacción.

El siguiente esquema resume el recorrido de la unidad:



## 1.1 Resultados de aprendizaje y criterios de evaluación de la UT2

**¿QUÉ DEBEMOS APRENDER DURANTE LA UNIDAD DE TRABAJO Y QUÉ SE VA A EVALUAR?**

| RESULTADO DE APRENDIZAJE: RA2 | | |
|---|---|---|
| **RA2.** Desarrolla aplicaciones que gestionan información almacenada en bases de datos relacionales identificando y utilizando mecanismos de conexión. | | |
| **CRITERIOS DE EVALUACIÓN** | | **%** |
| **a)** | Se han valorado las ventajas e inconvenientes de utilizar conectores. | **5%** |
| **b)** | Se han utilizado gestores de bases de datos embebidos e independientes. | **10%** |
| **c)** | Se ha utilizado el conector idóneo en la aplicación. | **10%** |
| **d)** | Se ha establecido la conexión. | **10%** |
| **e)** | Se ha definido la estructura de la base de datos. | **10%** |
| **f)** | Se han desarrollado aplicaciones que modifican el contenido de la base de datos. | **10%** |
| **g)** | Se han definido los objetos destinados a almacenar el resultado de las consultas. | **5%** |
| **h)** | Se han desarrollado aplicaciones que efectúan consultas. | **15%** |
| **i)** | Se han eliminado los objetos una vez finalizada su función. | **5%** |
| **j)** | Se han gestionado las transacciones. | **10%** |
| **k)** | Se han ejecutado procedimientos almacenados en la base de datos. | **10%** |

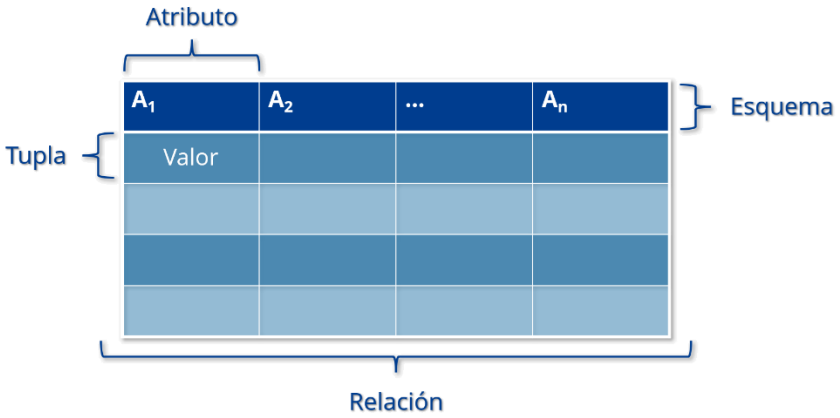
## 1.2 Conceptos previos

Actualmente, las bases de datos relacionales constituyen uno de los sistemas de almacenamiento más utilizados. Conviven con otras alternativas, como las bases de datos orientadas a objetos o las bases de datos documentales, que ofrecen soluciones diferentes para manejar grandes volúmenes de información o datos poco estructurados, y que estudiaremos en unidades de trabajo posteriores.

Las **bases de datos relacionales** presentan la información organizada en **tablas** compuestas de **filas y columnas**:

- Cada **columna** (también llamada **atributo** o **campo**) representa una característica de los datos que guarda la tabla y tiene un tipo de dato concreto: un número, un texto, una fecha…
- Cada **fila**, también llamada **tupla** o **registro**, representa un objeto de datos único dentro de la tabla, formado por un valor para cada columna.
- Cada fila se identifica de manera inequívoca mediante una **clave primaria** (*primary key*): una columna, o un grupo de columnas, cuyo valor no puede repetirse en dos filas de la misma tabla.

Esta estructura permite que los datos puedan consultarse y relacionarse entre sí de manera eficiente.




Veamos un ejemplo concreto. La siguiente tabla, CLIENTES, guarda los clientes de una empresa (la usaremos más adelante en los ejemplos de código de la unidad):

| DNI | APELLIDOS | CP |
|---|---|---|
| 78901234X | NADALES | 44126 |
| 56789012B | SAMPER | 29730 |
| 89012345E | HOJAS | (vacío: NULL) |

Cada fila es un cliente distinto y cada columna es un dato de ese cliente. La columna DNI es la clave primaria: no puede haber dos clientes con el mismo DNI, así que conociendo el DNI localizamos exactamente una fila. La columna CP (código postal) del último cliente no tiene valor; en SQL, la ausencia de valor se representa con NULL.

Las tablas, además, se **relacionan** entre sí mediante **claves ajenas** (*foreign keys*): una columna de una tabla que guarda el valor de la clave primaria de otra. Por ejemplo, una tabla FACTURAS podría tener una columna DNI_CLIENTE que indique a qué cliente de la tabla CLIENTES pertenece cada factura. Estos conceptos ya los conoces del módulo de Bases de Datos de primer curso; aquí solo los recordamos porque los vamos a manejar desde Java.

**¿Cómo se gestionan las bases de datos?**

Una aplicación no lee ni escribe directamente los ficheros en los que se guarda una base de datos. Entre la aplicación y los datos hay un programa especializado, el **sistema gestor de bases de datos** (**SGBD**), en inglés **Database Management System (DBMS)**. El SGBD se encarga de almacenar, mantener y recuperar la información: comprueba que los datos cumplen las reglas definidas (por ejemplo, que no se repita una clave primaria), controla qué usuarios pueden acceder a qué datos y permite que varios usuarios trabajen a la vez sin estropear la información. La aplicación, simplemente, le pide al SGBD lo que necesita y este le devuelve el resultado.

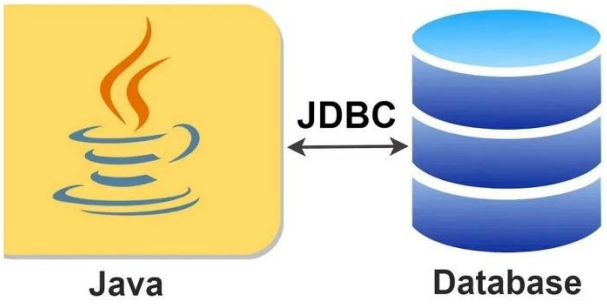
En el caso concreto de las bases de datos relacionales, el sistema gestor se denomina **Relational Database Management System (RDBMS)** o sistema gestor de bases de datos relacional. Existe una gran variedad de RDBMS en el mercado: MySQL, MariaDB, PostgreSQL, Oracle Database, Microsoft SQL Server, SQLite… Todos utilizan el lenguaje **SQL**, pero cada uno tiene sus propias características: su propia forma de comunicarse con las aplicaciones (su protocolo), sus propios tipos de datos y pequeñas diferencias en el SQL que admiten (lo que se conoce como su **dialecto** de SQL).

Por ejemplo, para obtener solo las diez primeras filas de una consulta, en MySQL se escribe LIMIT 10, en SQL Server TOP 10 y en Oracle FETCH FIRST 10 ROWS ONLY. Si cada aplicación tuviera que hablar directamente el “idioma" particular de su gestor, cambiar de gestor supondría reescribir buena parte del programa. Esta diversidad **complica la integración de las aplicaciones** con las bases de datos y ha dado lugar a lo que podría describirse como una “torre de Babel” en la programación de bases de datos.

Para simplificar el acceso a las bases de datos relacionales, **Java** ofrece una solución estandarizada: la API **Java Database Connectivity (JDBC)**. Esta interfaz permite que las aplicaciones escritas en Java se comuniquen de manera uniforme con distintos sistemas de bases de datos relacionales, sin necesidad de adaptar el código a cada sistema. JDBC se diseñó con tres objetivos clave:

1. **Compatibilidad con SQL**, para que las aplicaciones puedan construir y ejecutar sentencias SQL de forma directa.
2. **Aprovechamiento de APIs previas**, incorporando la experiencia de otros estándares de acceso a bases de datos que ya existían.
3. **Simplicidad**, haciendo que el acceso a bases de datos sea lo más directo y sencillo posible.

Con esta interfaz, Java facilita la interacción entre las aplicaciones y las bases de datos relacionales, superando las barreras técnicas que complicaban el desarrollo de software basado en datos. Antes de estudiar JDBC en detalle (apartado 3), en el apartado 2 analizaremos otro problema que aparece al combinar Java con una base de datos relacional: el desfase objeto-relacional.


JDBC
Java
Database

# 2 Desfase objeto-relacional

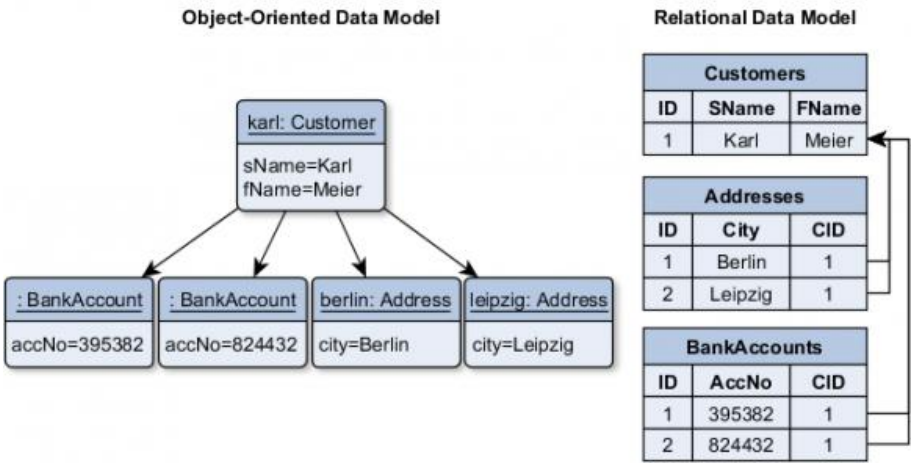
## 2.1 Origen del desfase objeto-relacional

El **desfase objeto-relacional**, también llamado **impedancia objeto-relacional** o **desajuste de impedancia**, es el conjunto de dificultades técnicas que aparecen al combinar una base de datos relacional con una aplicación escrita en un lenguaje de **programación orientada a objetos** (POO), como Java. Se debe a que ambos mundos organizan la información de forma diferente, y se manifiesta en tres aspectos:

**1. Diferencias en el lenguaje y el paradigma**

Una base de datos relacional guarda la información en **tablas** de filas y columnas, diseñadas a partir del modelo **Entidad-Relación** (E/R). Un programa Java, en cambio, trabaja con **clases** y **objetos**, que agrupan datos (atributos) y comportamiento (métodos). Para guardar o recuperar información hay que traducir continuamente de un modelo a otro:

- Una **tabla** equivale a una **clase**, y cada **fila** a un **objeto** de esa clase. Las **columnas** se corresponden con los **atributos**.
- En Java, un objeto se relaciona con otro guardando una **referencia** a él (por ejemplo, un objeto Cliente con una lista de objetos Cuenta). En el modelo relacional, esa relación se expresa con una **clave ajena** y, para recuperarla, hay que unir las tablas con un JOIN.



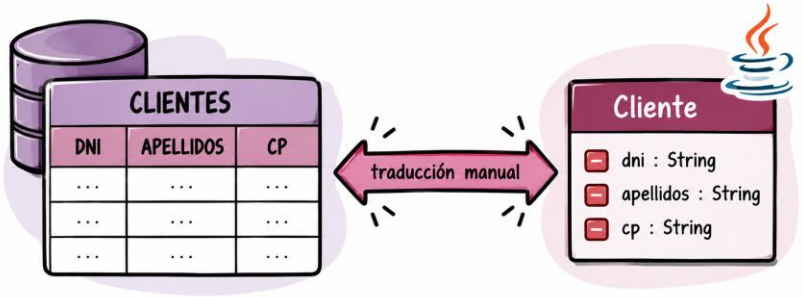

**2. Restricciones en los tipos de datos**

Las columnas de una tabla admiten tipos de datos simples: INT, VARCHAR, DATE… En Java, un atributo puede ser algo mucho más complejo: una lista de objetos, un array o una referencia a otro objeto que, a su vez, contiene otros. Una columna no puede guardar directamente una List<Cuenta>, así que el programador tiene que descomponer el objeto en varias filas (a menudo de varias tablas) al guardarlo y volver a montarlo al leerlo.

**3. Traducción entre modelos**

Como consecuencia de lo anterior, en un mismo proyecto hay que diseñar y mantener **dos modelos de datos**: el **diagrama Entidad-Relación**, que describe las tablas de la base de datos, y el **diagrama de clases**, que describe las clases de la aplicación. Esta dualidad

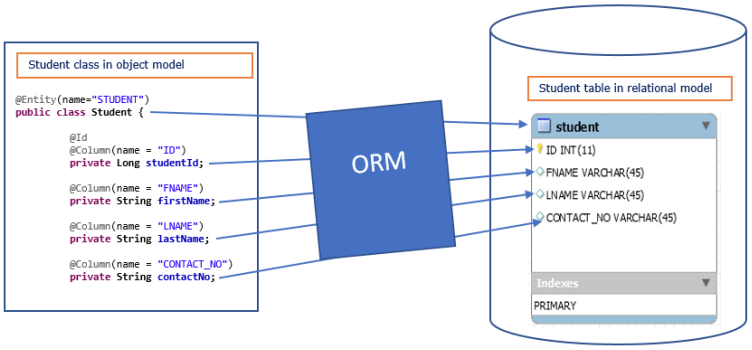
aumenta la complejidad del diseño y del desarrollo, y obliga a sincronizar los cambios: si se añade un dato nuevo, hay que modificar la tabla, la clase y el código que traduce entre ambas.




## 2.2 Soluciones al desfase objeto-relacional

En esta unidad haremos esa traducción "a mano" con los conectores, lo que nos permitirá entender qué ocurre por debajo. Existen, sin embargo, dos soluciones principales para reducir el desfase, que estudiaremos en las próximas unidades de trabajo:

- **Herramientas de mapeo objeto-relacional (ORM) – UNIDAD DE TRABAJO 3.** Establecen una correspondencia entre las clases del código y las tablas de la base de datos. El programador trabaja con objetos y la herramienta genera por él las sentencias SQL necesarias. En Java, la más conocida es **Hibernate**, que implementa el estándar **JPA**.

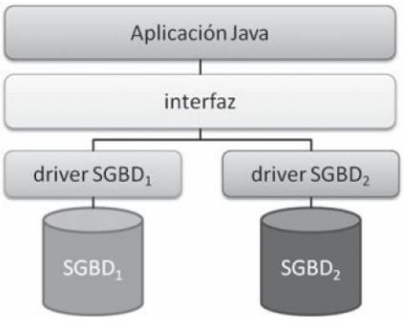



- **Bases de datos orientadas a objetos (BDOO) – UNIDAD DE TRABAJO 4.** Almacenan los objetos directamente, tal como existen en el programa, de modo que no hace falta traducirlos a tablas.

# 3 Protocolos de acceso a bases de datos: conectores

Cada sistema gestor de bases de datos (SGBD) se comunica con las aplicaciones mediante su propio **protocolo**, es decir, su propia forma de recibir peticiones y enviar respuestas por la red. Si una aplicación tuviera que hablar directamente el protocolo de cada gestor, el programador tendría que aprender uno distinto para MySQL, otro para Oracle, otro para PostgreSQL…

Para evitarlo existen **interfaces de alto nivel**: un conjunto de métodos estándar que sirven para trabajar con cualquier base de datos, sea cual sea su protocolo. La pieza que hace posible esta independencia es el **driver** (controlador). La aplicación siempre usa la misma interfaz común, y por debajo cada driver traduce esas llamadas al protocolo de un gestor concreto. Si cambiamos de gestor, cambiamos de driver, pero el código de la aplicación sigue siendo el mismo.




Estas interfaces de alto nivel permiten al programador:

- **Establecer una conexión** con una base de datos de forma sencilla.
- **Ejecutar sentencias SQL** sobre la base de datos.
- **Procesar los resultados** que devuelven las consultas.

Todas las operaciones se expresan en **SQL**, el lenguaje común a los gestores relacionales. El programador escribe la sentencia SQL, y el conector se encarga de llevarla hasta el gestor y de devolver el resultado a la aplicación, sin que haya que preocuparse de los detalles del protocolo.

**Al conjunto de clases que implementan la interfaz de programación de aplicaciones (API) y permiten el acceso a una base de datos se le denomina CONECTOR.** Una aplicación siempre necesita un conector para conectarse a una base de datos y lanzar consultas. El conector oculta los detalles específicos de cada gestor, de modo que el programador solo tiene que ocuparse de su aplicación. La mayoría de los fabricantes de bases de datos ofrecen conectores oficiales para sus productos.

## 3.1 Ejemplos más utilizados de conectores

Las dos arquitecturas de conectores más extendidas son **ODBC**, de Microsoft, y **JDBC**, creada por Sun Microsystems (hoy Oracle).

### 3.1.1 ODBC

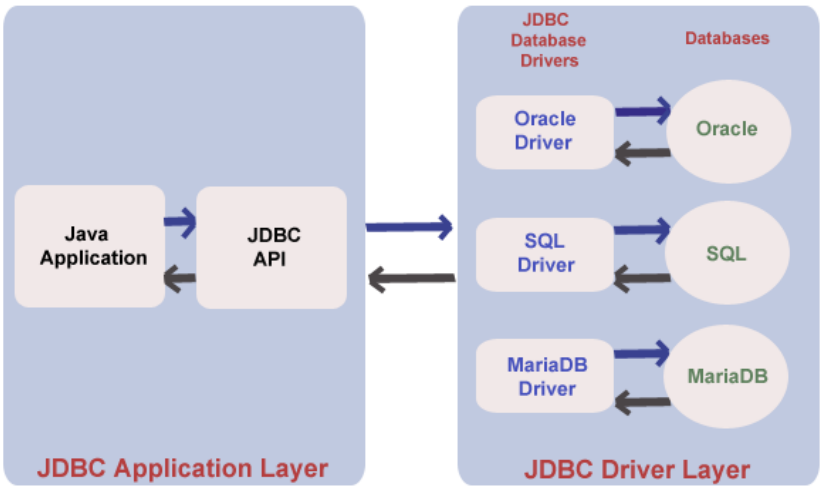
**ODBC** (*Open DataBase Connectivity*) fue la primera arquitectura de conectores. La desarrolló Microsoft para Windows a principios de los años noventa y es una API para el **lenguaje C**. Hoy se sigue usando ampliamente, tanto en Windows como en Linux y Unix: la mayoría de los gestores relacionales ofrecen un driver ODBC, y es la vía habitual para conectar, por ejemplo, una hoja de cálculo o una herramienta de informes con una base de datos.

En Java existió un **puente JDBC-ODBC** que convertía las llamadas JDBC en llamadas ODBC, para poder usar bases de datos que solo tenían driver ODBC. Este puente **se eliminó en Java 8 (2014)**, así que hoy no se puede utilizar.

### 3.1.2 JDBC

**JDBC** (*Java DataBase Connectivity*) surgió a finales de los años noventa como el equivalente a ODBC para Java. Es una capa de software intermedia que permite a los programas Java comunicarse con cualquier gestor relacional mediante SQL.

Su gran ventaja es que es **independiente de la plataforma y del gestor**: un mismo programa puede trabajar con MySQL, Oracle o PostgreSQL usando las mismas clases de Java. Para cambiar de gestor basta con cambiar el driver y los datos de conexión.




## 3.2 Conector JDBC

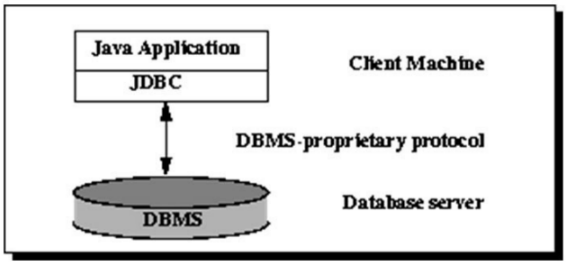
### 3.2.1 Arquitectura de conexión JDBC

Una aplicación Java puede acceder a la base de datos de dos formas, según cómo se organicen las piezas que intervienen: el **modelo de dos capas** y el **modelo de tres capas**.

**Modelo de dos capas (cliente-servidor)**

La aplicación se comunica **directamente** con la base de datos. Para ello necesita el driver JDBC del gestor concreto con el que trabaja. La aplicación envía las sentencias SQL a la base de datos, esta las procesa y le devuelve los resultados.

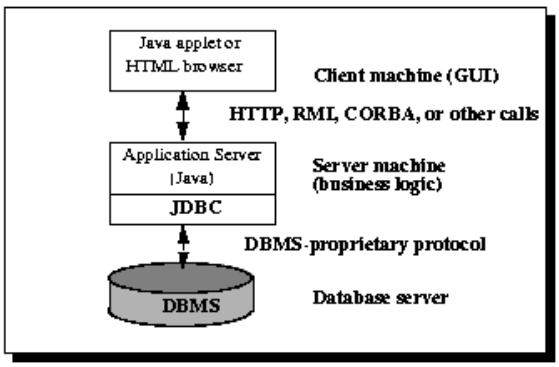
La base de datos puede estar en el mismo equipo que la aplicación o en otro, accesible por red. En este segundo caso, el equipo del usuario actúa como **cliente** y el equipo que aloja la base de datos actúa como **servidor**.




Es un modelo sencillo y eficiente, adecuado para aplicaciones de uso individual o con poco volumen de operaciones. Es el que usaremos en las prácticas de esta unidad.

**Modelo de tres capas**

Entre el cliente y la base de datos se añade una **capa intermedia** de servicios (por ejemplo, un servidor de aplicaciones). El cliente no envía las sentencias a la base de datos: hace sus peticiones a la capa intermedia, y es esta la que usa JDBC para acceder a los datos y devolverle el resultado.




Este modelo tiene varias ventajas:

- **Seguridad:** la base de datos no queda expuesta a los equipos de los usuarios, y las reglas de acceso se aplican en un único punto, la capa intermedia.
- **Escalabilidad:** la capa intermedia puede repartir la carga de trabajo y atender a muchos usuarios a la vez.
- **Separación de responsabilidades:** la lógica de negocio queda separada de la presentación y del acceso a datos, lo que facilita el mantenimiento.

Por eso es el modelo habitual en las aplicaciones empresariales y en las aplicaciones web.

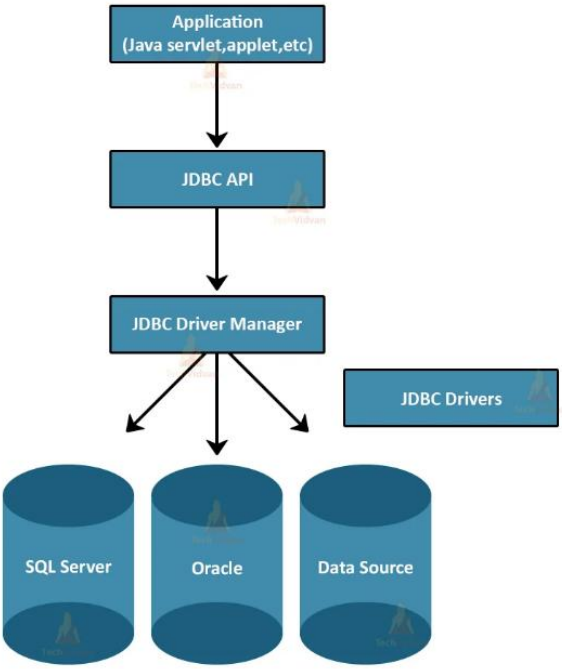
## 3.2.2 Componentes JDBC

JDBC se compone de varios elementos que trabajan juntos:

1. **API JDBC.** Es el conjunto de clases e interfaces que usamos en el código (Connection, Statement, ResultSet…). Forma parte de Java SE, así que no hay que instalar nada para utilizarla. Se reparte en dos paquetes:

- java.sql: las clases básicas para conectarse, ejecutar sentencias y procesar resultados. Es el que usaremos en esta unidad.
- javax.sql: funciones avanzadas, como las fuentes de datos (DataSource) y los *pools* de conexiones, que reutilizan conexiones abiertas para mejorar el rendimiento en aplicaciones con muchos usuarios.

2. **Gestor de drivers (DriverManager).** Es la clase que se encarga de encontrar el driver adecuado. Cuando le pedimos una conexión, busca entre los drivers disponibles en el proyecto uno que sepa conectar con la base de datos indicada y nos devuelve la conexión abierta.
3. **Drivers JDBC.** Son las librerías que implementan la API para un gestor concreto. No vienen con Java: los proporciona cada fabricante y hay que añadirlos al proyecto (por ejemplo, *MySQL Connector/J* para MySQL o *sqlite-jdbc* para SQLite).
4. **Suite de pruebas JDBC.** Es un conjunto de pruebas que permite comprobar que un driver cumple el estándar JDBC.

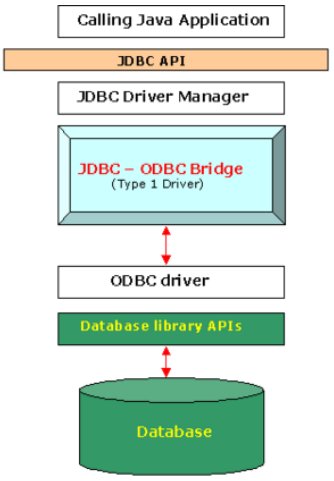



### 3.2.3 Tipos de conectores JDBC

Los drivers JDBC se clasifican en **cuatro tipos**, numerados del 1 al 4, según cómo están construidos y cómo llegan hasta la base de datos. La clasificación ayuda a elegir el driver más adecuado y explica por qué hoy se usa casi exclusivamente el tipo 4.

**Conector tipo 1: puente JDBC-ODBC**

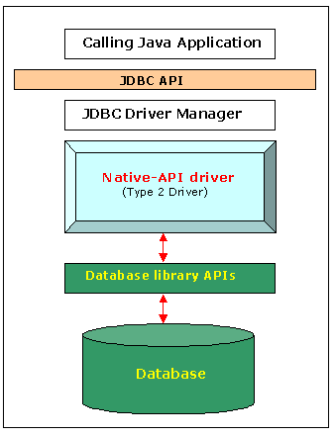
Traduce las llamadas JDBC a llamadas ODBC, apoyándose en librerías del sistema operativo. Permitía conectarse a bases de datos que solo tenían driver ODBC.




**Vigencia:** se eliminó del JDK en Java 8 (2014). Hoy solo tiene interés histórico.

### Conector tipo 2: driver parcialmente nativo

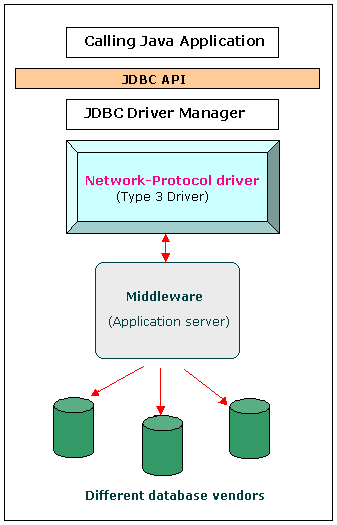
Es un driver híbrido: una parte está escrita en Java y otra en código nativo del sistema operativo. Traduce las llamadas JDBC a llamadas a la librería cliente del fabricante, que debe estar instalada en el equipo. Por eso no es un driver completamente Java.




**Vigencia:** su uso es residual, porque obliga a instalar librerías nativas en cada equipo cliente.

### Conector tipo 3: driver middleware (Java puro)

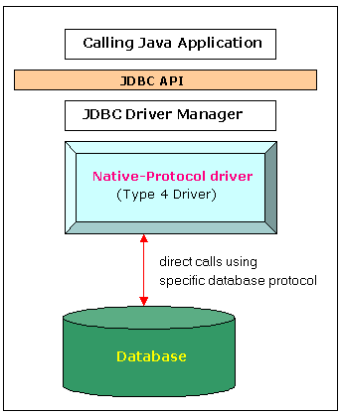
El driver está escrito completamente en Java, pero no habla directamente con la base de datos: se comunica por red (TCP/IP) con un **servidor intermedio** (*middleware*), y es este el que traduce las peticiones al protocolo del gestor.




**Vigencia:** es poco habitual, porque añade un servidor intermedio que hay que instalar y mantener.

**Conector tipo 4: driver de protocolo nativo**

Lo proporciona el propio fabricante del gestor y está escrito completamente en Java. Convierte las llamadas JDBC directamente en el protocolo de red del gestor, sin intermediarios. Es independiente de la plataforma y es el que ofrece mejor rendimiento.




**Vigencia:** es el tipo que usaremos. **MySQL Connector/J** es un driver de tipo 4.

## 3.3 Ventajas e inconvenientes de los conectores

Trabajar directamente con un conector como JDBC tiene puntos fuertes y limitaciones que conviene conocer:

| Ventajas | Inconvenientes |
|---|---|
| **Independencia del gestor:** el mismo código sirve para MySQL, PostgreSQL u Oracle cambiando el driver y la URL de conexión. | **El SQL va dentro de cadenas de texto:** el compilador no lo revisa, así que los errores en una sentencia aparecen al ejecutar el programa, no al compilarlo. |

| Ventajas | Inconvenientes |
|---|---|
| **Es estándar:** la API forma parte de Java SE. | **La independencia no es total:** cada gestor tiene su propio dialecto SQL, y algunas sentencias hay que adaptarlas al cambiar de gestor. |
| **Control total del SQL** que se ejecuta. | **No resuelve el desfase objeto-relacional:** hay que convertir a mano las filas en objetos y viceversa. |
| **Es la base de los ORM (UT2): herramientas como Hibernate usan JDBC por debajo.** | **Código repetitivo:** abrir y cerrar recursos y gestionar excepciones se repite en cada operación. |

## 3.4 Elección del conector idóneo

Para elegir el conector de una aplicación hay que seguir tres criterios:

1. **Que corresponda al gestor** que vamos a usar:
   - MySQL → MySQL Connector/J.
   - MariaDB → MariaDB Connector/J.
   - SQLite → sqlite-jdbc.
2. **Preferir un driver de tipo 4 del propio fabricante**, por rendimiento y porque no requiere instalar nada más en el equipo.
3. **Que su versión sea compatible** con la del servidor de base de datos y con la versión de Java del proyecto (en nuestro caso, JDK 21).

En esta unidad usaremos **MySQL Connector/J** para conectarnos a MySQL y **sqlite-jdbc** para SQLite.

# 4 Establecimiento de conexiones

Antes de hacer cualquier operación sobre una base de datos, la aplicación tiene que **conectarse** a ella. En JDBC esto se hace con el método getConnection() de la clase DriverManager, que recibe una **URL JDBC** que identifica la base de datos y, si el gestor lo requiere, un usuario y una contraseña.

DriverManager recorre los drivers disponibles en el proyecto hasta encontrar uno que reconozca esa URL. Si lo encuentra y los datos son correctos, devuelve un objeto Connection que representa la conexión abierta. Si no encuentra ningún driver adecuado, o la conexión falla, lanza una SQLException.

Una conexión ocupa recursos tanto en la aplicación como en el servidor, así que hay que **cerrarla** cuando ya no se necesita. Esta es la forma clásica de hacerlo, con un bloque finally, que se ejecuta siempre, haya error o no:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionClasica {

    // Datos de conexion
    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        Connection connection = null;
        try {
            // Establecer la conexion con la base de datos MySQL
            connection = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("Conexion establecida correctamente.");
        } catch (SQLException e) {
            System.out.println("Error al conectar a la base de datos.");
            e.printStackTrace();
        } finally {
            // Cerrar la conexion (se ejecuta siempre, haya error o no)
            if (connection != null) {
                try {
                    connection.close();
                    System.out.println("Conexion cerrada.");
                } catch (SQLException ex) {
                    ex.printStackTrace();
                }
            }
        }
    }
}
```

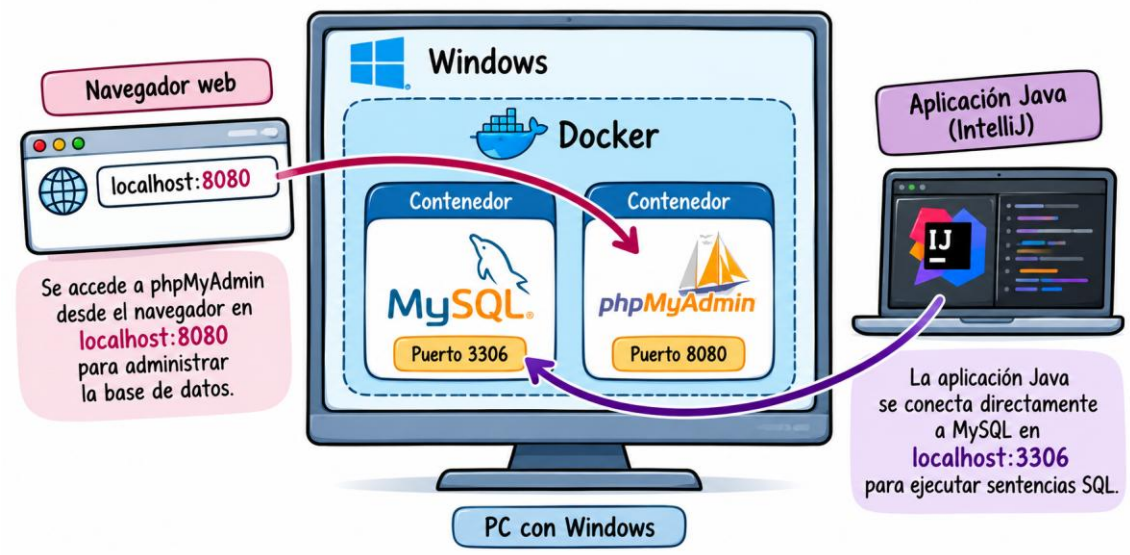
Como ves, cerrar la conexión de esta forma exige bastante código. En el apartado 4.3 veremos una manera más cómoda y segura de hacerlo. Pero antes, necesitamos dos cosas: una base de datos a la que conectarnos y el driver.

## 4.1 Preparación de la base de datos: MySQL con Docker

En esta unidad accederemos desde Java a un servidor **MySQL** que se ejecuta dentro de un **contenedor Docker**. El entorno se levanta con **Docker Compose** a partir de un fichero docker-compose.yml (lo tienes en el Aula Virtual) y tiene dos servicios:

- **MySQL** (servicio db): el gestor de bases de datos.

- **phpMyAdmin**: una aplicación web para administrar MySQL desde el navegador, en http://localhost:8080.




Para preparar el entorno, sigue estos pasos:

1. Arranca **Docker Desktop** y espera a que indique que está en funcionamiento.
2. Abre el fichero docker-compose.yml y, en el servicio db, **descomenta las líneas ports** (borra el carácter # del principio de esas líneas). Así se **publica el puerto 3306**: MySQL escucha en el puerto 3306 dentro del contenedor, y publicarlo hace que también sea accesible desde Windows como localhost:3306. phpMyAdmin no lo necesita porque se comunica con MySQL desde dentro de Docker, pero nuestro programa Java se ejecuta fuera.
3. Abre un terminal en la carpeta donde está el fichero (puede ser el terminal de IntelliJ) y ejecuta docker compose up -d. La opción -d arranca los contenedores en segundo plano y te devuelve el control del terminal. Puedes comprobar que están en marcha con docker compose ps o desde Docker Desktop.
4. Entra en phpMyAdmin (http://localhost:8080) y **crea las tablas** que vayas a usar dentro de la base de datos db, desde la pestaña SQL.

Los datos de conexión que usaremos en todos los ejemplos son:

| Dato | Valor |
|---|---|
| **Servidor y puerto** | localhost:3306 |
| **Base de datos** | db |
| **Usuario** | dam |
| **Contraseña** | 123456 |

**Nota:** la base de datos db ya la crea Docker Compose al arrancar, así que no hay que crearla. En general, crear una base de datos (CREATE DATABASE) no forma parte del SQL estándar y cada gestor lo hace a su manera, por lo que suele ser una tarea del administrador y no del programa Java.

**Importante:** si al arrancar aparece un error indicando que el puerto 3306 ya está en uso, es que hay otro MySQL funcionando en el equipo (por ejemplo, el de XAMPP). Detenlo y vuelve a ejecutar docker compose up -d.

## 4.2 Driver de conexión y proyecto Maven

MySQL ofrece conectores para varios lenguajes. Para Java, el driver oficial es **MySQL Connector/J**, un driver JDBC de **tipo 4**: está escrito completamente en Java y habla directamente el protocolo de MySQL, sin necesitar ninguna librería de MySQL instalada en el equipo.

El driver es una librería externa (un fichero .jar) que hay que añadir al proyecto. Hay dos formas de hacerlo en IntelliJ:

- **Añadir la librería manualmente:** descargar el .jar del conector y añadirlo desde *File → Project Structure → Libraries*. Es sencillo, pero hay que repetirlo en cada proyecto y actualizarlo a mano.
- **Crear el proyecto con Maven y añadir la dependencia (recomendado):** indicamos qué librería necesitamos y Maven la descarga y la incorpora al proyecto automáticamente.

[VÍDEO: configuración de la API JDBC en IntelliJ]

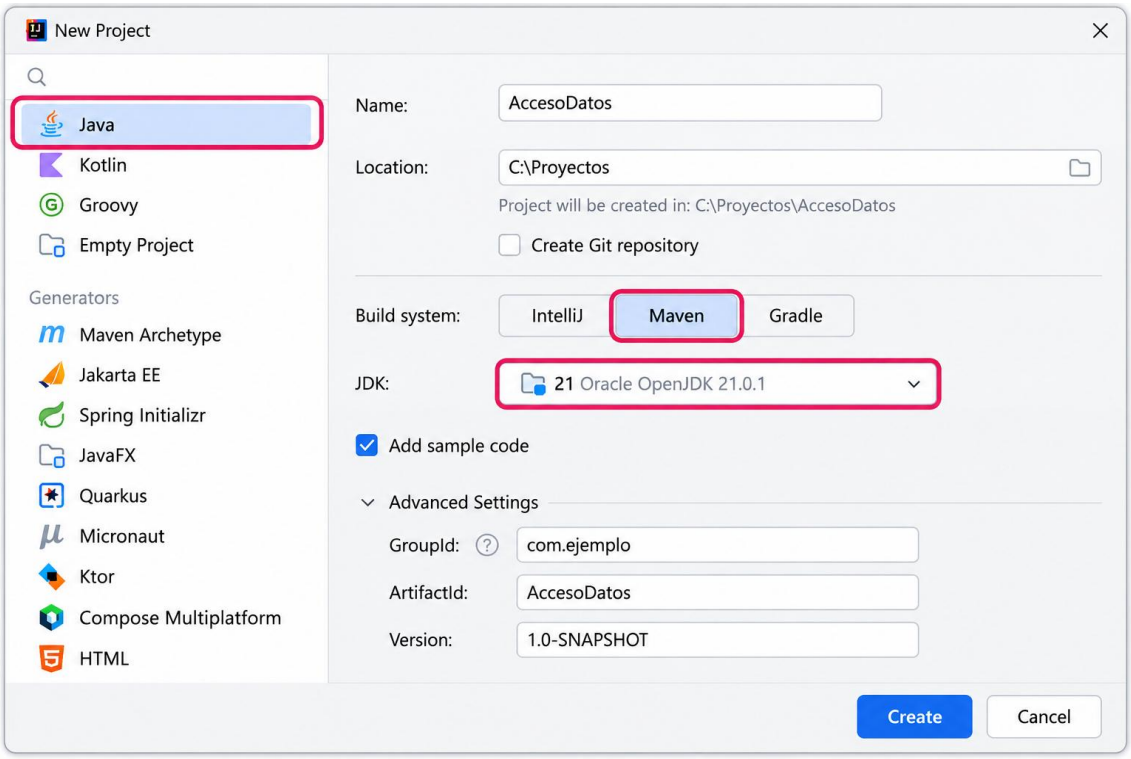
Los pasos con Maven son:

1. En IntelliJ, crea el proyecto con *File → New → Project*, elige **Java**, en *Build system* selecciona **Maven** y como JDK la versión **21**.
2. Abre el fichero pom.xml que se ha generado en la raíz del proyecto y añade la dependencia del conector dentro de la etiqueta <dependencies> (si no existe, créala justo antes de </project>):

```
<dependencies>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>9.7.0</version>
    </dependency>
</dependencies>
```

A continuación, pulsa el botón *Load Maven Changes* que aparece en el editor (o **Ctrl + Mayús + O**) para que Maven descargue el driver. Lo verás en el panel *Maven*, dentro de *Dependencies*.

[ENLACE: página de MySQL Connector/J en el repositorio de Maven]




**Nota:** en programas antiguos verás la línea Class.forName("com.mysql.cj.jdbc.Driver") antes de conectar. Servía para cargar el driver, pero desde JDBC 4.0 (Java 6) los drivers se cargan solos al estar en el proyecto, así que no hace falta escribirla.

## 4.3 Apertura de conexiones

La interfaz Connection representa el canal de comunicación entre la aplicación y la base de datos. Para obtener un objeto Connection se usa el método getConnection() de DriverManager, que tiene tres versiones (sobrecargas):

```
getConnection(String url);
getConnection(String url, Properties info);
getConnection(String url, String user, String password);
```

La más habitual es la tercera, que recibe la URL, el usuario y la contraseña. La primera se usa con gestores que no piden credenciales (como SQLite) y la segunda permite pasar el usuario, la contraseña y otras opciones dentro de un objeto Properties.

La **URL JDBC** tiene una estructura fija. Para nuestra base de datos MySQL es:

```
jdbc:mysql://localhost:3306/db
```

| Parte | Significado |
|---|---|
| **jdbc** | Protocolo. Todas las URL JDBC empiezan así. |
| **mysql** | Subprotocolo: indica el gestor y, por tanto, qué driver debe usarse. |
| **localhost:3306** | Servidor y puerto donde escucha el gestor. |

| Parte | Significado |
|---|---|
| **db** | Nombre de la base de datos. |

Desde Java 7, la forma recomendada de abrir una conexión es declararla dentro de un bloque **try-with-resources**: los objetos que se crean entre los paréntesis del try se cierran solos al salir del bloque, tanto si todo va bien como si se produce un error. Así nos aseguramos de que los objetos se eliminan una vez terminada su función, con mucho menos código:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionMySQL {

    // Datos de conexion del contenedor Docker del aula
    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        // try-with-resources: la conexion se cierra sola al salir del bloque
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD)) {
            System.out.println("Conectado a: "
                    + connection.getMetaData().getDatabaseProductName() + " "
                    + connection.getMetaData().getDatabaseProductVersion());
        } catch (SQLException e) {
            System.out.println("Error al conectar: " + e.getMessage());
        }
    }
}
```

El método getMetaData() devuelve información sobre la base de datos a la que estamos conectados. Aquí lo usamos para mostrar el nombre y la versión del gestor, lo que confirma que la conexión funciona:

```
Conectado a: MySQL 8.0.46
```

**Importante:** try-with-resources cierra la conexión automáticamente, incluso si se produce un error. Es la forma que usaremos en el resto de la unidad.

Si la conexión falla, el mensaje de la excepción suele indicar la causa:

| Mensaje | Causa habitual |
|---|---|
| ***Communications link failure*** | El servidor no está arrancado o el puerto 3306 no está publicado (revisa el apartado 4.1). |
| ***Access denied for user*** | El usuario o la contraseña no son correctos. |
| ***No suitable driver found*** | El driver no está en el proyecto (revisa el pom.xml) o la URL está mal escrita. |

## 4.4 Gestores embebidos e independientes

Hasta ahora hemos trabajado con un **gestor independiente**: MySQL es un programa que se ejecuta por su cuenta (en nuestro caso, en un contenedor) y al que las aplicaciones se conectan por red. Existe otro tipo de gestores, los **embebidos**, que no son un programa aparte: son una librería que se ejecuta dentro de la propia aplicación y guarda los datos en un fichero local.

| | Independiente (servidor) | Embebido |
|---|---|---|
| **Dónde se ejecuta** | Proceso aparte (nuestro contenedor Docker) | Dentro de la propia aplicación Java |
| **Conexión** | Por red: servidor y puerto | Fichero local o memoria |
| **Usuarios a la vez** | Muchos | Normalmente uno |
| **Instalación** | Servidor + driver | Solo una dependencia Maven |
| **Ejemplos** | MySQL, PostgreSQL, Oracle | SQLite, H2, Apache Derby |

Un gestor independiente es la opción adecuada cuando varios usuarios o aplicaciones comparten los mismos datos. Un gestor embebido es ideal cuando los datos pertenecen a una sola aplicación: no hay que instalar ni arrancar ningún servidor. **SQLite** es el gestor embebido más utilizado del mundo: está en Android, en los navegadores web y en millones de aplicaciones.

**Conexión a un gestor embebido: SQLite**

Para usar SQLite basta con añadir su driver al pom.xml:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.53.2.1</version>
</dependency>
```

El código de conexión es el mismo que con MySQL. Solo cambia la URL, que ahora indica el fichero de la base de datos, y no hay usuario ni contraseña:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;


public class ConexionSQLite {


    public static void main(String[] args) {
        // Crea (si no existe) el fichero agenda.db en la carpeta del proyecto
        try (Connection con = DriverManager.getConnection("jdbc:sqlite:agenda.db")) {
            String gestor = con.getMetaData().getDatabaseProductName();
            System.out.println("Conectado a: " + gestor);
        } catch (SQLException e) {
            System.out.println("Error al conectar: " + e.getMessage());
        }
    }
}
```

Si el fichero agenda.db no existe, SQLite lo crea vacío. Al ejecutar desde IntelliJ, se crea en la carpeta raíz del proyecto. También se puede indicar una ruta completa (jdbc:sqlite:C:/datos/agenda.db) o crear una base de datos temporal en memoria (jdbc:sqlite::memory:), que desaparece al cerrar la conexión.

> **Nota:** como SQLite no tiene servidor, no se administra con phpMyAdmin. Las tablas se crean desde el propio programa Java, con las sentencias que veremos en el apartado 5.2.

## 4.5 Realización de consultas

Una vez abierta la conexión, cualquier programa que trabaje con la base de datos sigue los mismos pasos:

1. **Establecer la conexión** con la base de datos (Connection).
2. **Enviar la sentencia SQL** (Statement) y **procesar el resultado**.
3. **Liberar los recursos** al terminar, cerrando los objetos que ya no se necesitan.
4. **Gestionar los errores** que puedan surgir (SQLException).

Las sentencias que enviamos pueden ser de dos tipos:

- **Consultas (SELECT)**: devuelven datos en forma de filas. Se ejecutan con executeQuery(), que devuelve un objeto ResultSet con esas filas. Se recorre fila a fila con el método next().
- **Actualizaciones (INSERT, UPDATE, DELETE)**: modifican los datos. Se ejecutan con executeUpdate().

Por ejemplo, partiendo de esta tabla:

```
CREATE TABLE usuarios (
    id     INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    edad   INT
);
INSERT INTO usuarios VALUES (1, 'Ana', 21), (2, 'Luis', 19), (3, 'Marta', 23);
```

este programa muestra todos sus usuarios:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConsultaUsuarios {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        String sql = "SELECT id, nombre, edad FROM usuarios";

        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(sql)) {

            // Recorrer el resultado fila a fila
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String nombre = resultSet.getString("nombre");
                int edad = resultSet.getInt("edad");
                System.out.println("ID: " + id + ", Nombre: " + nombre
                        + ", Edad: " + edad);
            }
        } catch (SQLException e) {
            System.out.println("Error al realizar la consulta: " + e.getMessage());
        }
    }
}
```

Fíjate en que el try abre tres recursos: la conexión, la sentencia y el resultado. Al salir del bloque se cierran los tres, en orden inverso a su apertura. La salida es:

```
ID: 1, Nombre: Ana, Edad: 21
ID: 2, Nombre: Luis, Edad: 19
ID: 3, Nombre: Marta, Edad: 23
```

En el apartado 5 veremos con detalle cada tipo de sentencia.

# 5 Operaciones básicas

En este apartado veremos cómo ejecutar con JDBC los principales tipos de sentencias SQL. Recuerda que SQL se divide en varios sublenguajes, y que cada tipo de sentencia se ejecuta en JDBC con un método distinto:

- **DML** (*Data Manipulation Language*, lenguaje de manipulación de datos), que incluye dos tipos de operaciones:
  - **Operaciones de consulta** (SELECT): se ejecutan con executeQuery(), que devuelve las filas obtenidas en un ResultSet.
  - **Operaciones de modificación de datos** (INSERT, UPDATE y DELETE): se ejecutan con executeUpdate(), que devuelve el número de filas afectadas.
- **DDL** (*Data Definition Language*, lenguaje de definición de datos): sentencias como CREATE, ALTER o DROP, que definen la estructura de la base de datos. Se ejecutan con execute().

El siguiente esquema resume todo lo necesario para ejecutar cualquier sentencia SQL con JDBC:

| | | |
|---|---|---|
| `Class.forName(nombre del driver); // No necesario desde JDBC 4.0 (Java SE 6)` | | Cargar *driver* |
| `Connection c = DriverManager.getConnection(datos de conexión);` | | Crear conexión |
| `Statement s = c.createStatement();` | | Crear sentencia |
| `ResultSet rs = s.executeQuery(consulta);`<br>`while(rs.next()) {`<br>`  ...`<br>`}`<br>`rs.close();` | `int res = s.executeUpdate (sent. DML);`<br>(o bien)<br>`boolean res = s.execute (sent. DDL);` | Ejecutar sentencia<br>Si consulta, obtener resultados fila a fila y cerrar lista de resultados |
| `s.close();` | | Cerrar sentencia |
| `c.close();` | | Cerrar conexión |

La primera fila del esquema (Class.forName) ya no es necesaria, como vimos en el apartado 4.2. Y si usamos try-with-resources, las llamadas a close() de las últimas filas tampoco hay que escribirlas.

## 5.1 La interfaz Statement

La interfaz Statement es la herramienta básica de JDBC para ejecutar sentencias SQL. Con ella se pueden ejecutar consultas SELECT, sentencias DML (INSERT, UPDATE y DELETE) y sentencias DDL (CREATE, ALTER…). Un objeto Statement se obtiene a partir de la conexión, con el método createStatement():

```
Statement s = connection.createStatement();
```

Sus métodos principales son:

| MÉTODO | FUNCIONALIDAD |
|---|---|
| **ResultSet executeQuery(String sql)** | Ejecuta una consulta SELECT y devuelve un ResultSet con los resultados. |
| **int executeUpdate(String sql)** | Ejecuta una sentencia que modifica los datos (INSERT, UPDATE o DELETE). Devuelve el número de filas afectadas. |
| **boolean execute(String sql)** | Ejecuta cualquier tipo de sentencia. Es el método que se usa para las sentencias DDL (CREATE, ALTER, DROP).<br>Devuelve true si la sentencia ha producido un ResultSet (una consulta) y false si no (por ejemplo, una sentencia DDL). |
| **ResultSet getResultSet()** | Obtiene el ResultSet de una consulta lanzada con execute(). También se usa con los procedimientos almacenados (apartado 7). |

### Ejemplo con errores

En JDBC, cada sentencia SQL se escribe dentro de una cadena de texto. Esto tiene una consecuencia importante, que ya señalamos entre los inconvenientes de los conectores: **el compilador de Java no revisa el SQL**. Observa este fragmento, que contiene dos errores:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;

public class JDBC_create  table {
    public static void main(String[] args) {
        (...) // Se omite la declaracion de variables para los datos de conexion
        try (Connection c = DriverManager.getConnection(urlconnection, user, pwd);
             Statement s = c.createStatement()) {
            s.execute("CREATE TABLE CLIENTES (DNI CHAR(9)NOT NULL, APELLIDOS VARCHAR(32)
NOT NULL, CP CHAR(5),PRIMARYKEY(DNI))");
            //...
        } catch (SQLException e) {
            muestraErrorSQL(e);
        }
    }
}
```

1. El nombre de la clase, JDBC_create table, contiene un espacio. Este error es de Java, así que lo detecta el compilador y el programa ni siquiera llega a ejecutarse.
2. En la sentencia SQL falta un espacio en PRIMARYKEY. Para Java es solo una cadena de texto, así que el programa compila sin problemas. El error aparece **al ejecutar**: MySQL recibe la sentencia, no la entiende y JDBC lanza una SQLException.

Por eso conviene probar cada sentencia SQL (por ejemplo, en phpMyAdmin) antes de escribirla en el programa. En el siguiente apartado veremos la versión correcta.

## 5.2 Ejecución de sentencias de descripción de datos (DDL)

El **lenguaje de definición de datos (DDL)** permite crear, modificar y eliminar la estructura de la base de datos: tablas, vistas, índices, etc. Sus sentencias (CREATE, ALTER y DROP) se ejecutan con el método execute().

Este programa crea la tabla CLIENTES que vimos en el apartado 1.2:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class CrearTablaClientes {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        try (Connection c = DriverManager.getConnection(URL, USER, PASSWORD);
             Statement s = c.createStatement()) {

            s.execute("CREATE TABLE CLIENTES ("
                    + "DNI CHAR(9) NOT NULL, "
                    + "APELLIDOS VARCHAR(32) NOT NULL, "
                    + "CP CHAR(5), "
                    + "PRIMARY KEY (DNI))");
            System.out.println("Tabla CLIENTES creada.");

        } catch (SQLException e) {
            muestraErrorSQL(e);
        }
    }

    private static void muestraErrorSQL(SQLException e) {
        System.err.println("Mensaje: " + e.getMessage());
        System.err.println("SQLState: " + e.getSQLState());
        System.err.println("Codigo de error: " + e.getErrorCode());
    }
}
```

La conexión y la sentencia se declaran en el try-with-resources, así que no hace falta llamar a close(). Si ejecutas el programa dos veces, la segunda vez fallará, porque la tabla ya existe. El método muestraErrorSQL() muestra entonces algo así:

```
Mensaje: Table 'CLIENTES' already exists
SQLState: 42S01
Codigo de error: 1050
```

Estos son los tres datos que ofrece una SQLException:

- getMessage(): la descripción del error.
- getSQLState(): un código estándar de cinco caracteres que indica el tipo de error. Es el mismo en todos los gestores.
- getErrorCode(): el código de error propio del fabricante. En MySQL, el 1050 significa "la tabla ya existe".

**Consejo:** para que el programa pueda ejecutarse varias veces sin error, se puede escribir CREATE TABLE IF NOT EXISTS CLIENTES (...). Así, si la tabla ya existe, MySQL no hace nada.

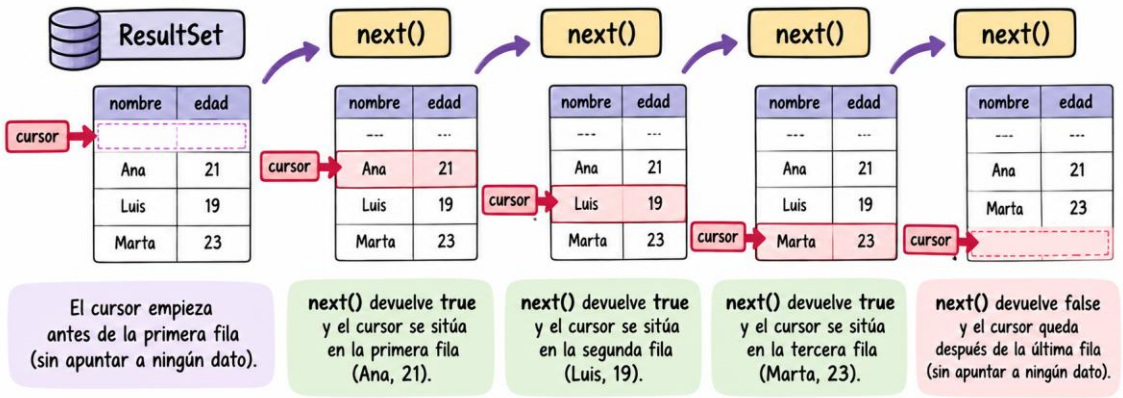
## 5.3 Ejecución de consultas (DML - consultas)

### 5.3.1 executeQuery

Las **consultas** son las sentencias SELECT. Se ejecutan con el método executeQuery(), que devuelve un objeto ResultSet con el resultado: un conjunto de filas organizadas en columnas, como una tabla.

**ResultSet y el cursor de navegación**

Un ResultSet no nos entrega todas las filas a la vez. Tiene un **cursor**, una especie de puntero que indica cuál es la **fila actual**. Al principio, el cursor está situado **antes de la primera fila**. Cada llamada a next() lo avanza a la fila siguiente, y entonces podemos leer los valores de esa fila con los métodos getXXX(). Cuando ya no quedan filas, next() devuelve false. Por eso los resultados se recorren con un bucle while (rs.next()).




| MÉTODO | FUNCIONALIDAD |
|---|---|
| **boolean next()** | Mueve el cursor a la fila siguiente. Devuelve true si hay fila y false si ya se han recorrido todas. |
| **getXXX(int columna)**<br>**getXXX(String columna)** | Devuelve el valor de una columna de la fila actual. La columna se indica por su posición (**empezando en 1**) o por su nombre. Hay un método para cada tipo: getInt(), getString(), getDouble(), getBigDecimal(), getDate()… |
| **void close()** | Cierra el ResultSet. Debe hacerse siempre (con try-with-resources se hace solo). |

**Ejecución básica de una consulta SELECT**

El patrón es siempre el mismo: crear el Statement, llamar a executeQuery() y recorrer el ResultSet. El programa completo lo viste en el apartado 4.5; esta es la parte que hace la consulta:

```
try (Statement statement = connection.createStatement();
     ResultSet resultSet = statement.executeQuery("SELECT nombre, edad FROM usuarios")) {

    while (resultSet.next()) {
        String nombre = resultSet.getString("nombre");
        int edad = resultSet.getInt("edad");
        System.out.println("Nombre: " + nombre + ", Edad: " + edad);
    }
}
```

## 5.3.2 Sentencias preparadas

Hasta ahora, las sentencias SQL eran fijas. En una aplicación real lo normal es que dependan de **variables**: el DNI que el usuario escribe en un buscador, los datos de un formulario… Una primera idea sería construir la sentencia concatenando la variable:

```
String consulta = "SELECT * FROM CLIENTES WHERE DNI='" + dni + "'";
```

Esta forma de trabajar tiene dos problemas importantes:

1. **Seguridad.** Es vulnerable a la **inyección de SQL**: si el usuario escribe en el buscador el texto ' OR '1'='1, la sentencia resultante es SELECT * FROM CLIENTES WHERE DNI='' OR '1'='1'. Como '1'='1' siempre es cierto, la consulta devuelve todos los clientes. Con textos más elaborados, un atacante podría incluso modificar o borrar datos.
2. **Rendimiento.** Cada vez que el SGBD recibe una sentencia, tiene que analizarla y calcular un **plan de ejecución** (la forma más rápida de resolverla). Si la sentencia cambia en cada ejecución, aunque solo sea el DNI, el SGBD repite ese trabajo cada vez.

**Ventajas de las sentencias preparadas**

Las **sentencias preparadas** (PreparedStatement) resuelven ambos problemas:

- **Marcadores de parámetros** (*placeholders*): en lugar de concatenar, se escribe la sentencia con un ? en cada lugar donde va un valor variable.
- **Envío único:** la sentencia se envía al servidor una sola vez y se compila. A partir de ahí, su estructura ya no puede cambiar: los valores que se pongan después en los ? se tratan siempre como datos, nunca como parte del SQL. Así se evita la inyección.
- **Reutilización del plan:** el plan de ejecución se reutiliza en cada ejecución; solo cambian los valores.

**Nota:** MySQL Connector/J, por defecto, no envía la sentencia al servidor para compilarla: la prepara en el propio driver, que coloca los valores en los ? de forma segura. La protección frente a la inyección de SQL es la misma. Para que el servidor compile la sentencia y reutilice el plan, hay que añadir ?useServerPrepStmts=true al final de la URL de conexión.

### Uso de PreparedStatement

Trabajar con una sentencia preparada tiene tres pasos:

1. **Definir la sentencia SQL** con un ? por cada valor variable. El ? no lleva comillas, aunque el valor sea un texto:

```
String sql = "SELECT * FROM CLIENTES WHERE DNI = ?";
```

2. **Obtener el PreparedStatement** con el método prepareStatement() de Connection. En este momento la sentencia queda "impresa" y ya nadie puede cambiarla:

```
PreparedStatement ps = connection.prepareStatement(sql);
```

3. **Rellenar los marcadores** con los métodos setXXX(posición, valor) (el primer ? es la posición 1) y ejecutar. Fíjate en que ahora executeQuery() no recibe la sentencia, porque ya la tiene:

```
ps.setString(1, dni);
ResultSet rs = ps.executeQuery();
```

Los métodos propios de PreparedStatement son:

| MÉTODO | FUNCIONALIDAD |
|---|---|
| **ResultSet executeQuery()**<br>**int executeUpdate()**<br>**boolean execute()** | Igual que en Statement, pero sin parámetro: la sentencia se indicó al crear el objeto. |
| **setXXX(int pos, YYY valor)** | Asigna un valor al ? de la posición pos (el primero es el 1). Hay uno por tipo: setString(), setInt(), setDouble(), setBigDecimal(), setDate()… |
| **setNull(int pos, int tipoSQL)** | Asigna el valor NULL. Hay que indicar el tipo SQL de la columna con una constante de la clase java.sql.Types, por ejemplo Types.INTEGER. |

**Ejemplo de uso de PreparedStatement**

Este programa inserta dos clientes en la tabla CLIENTES1, reutilizando la misma sentencia preparada. La tabla se crea así:

```
CREATE TABLE CLIENTES1 (
    DNI       CHAR(9) NOT NULL,
    APELLIDOS VARCHAR(32) NOT NULL,
    CP        INTEGER,
    PRIMARY KEY (DNI)
);
```

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;


public class InsertarClientesPreparada {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        // Sentencia con marcadores de posicion (?)
        String sql = "INSERT INTO CLIENTES1 (DNI, APELLIDOS, CP) VALUES (?, ?, ?)";

        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            // Primer cliente
            int i = 1;
            ps.setString(i++, "123456789");
            ps.setString(i++, "Garcia");
            ps.setInt(i++, 28001);
            ps.executeUpdate();

            // Segundo cliente: se reutiliza la misma sentencia preparada
            i = 1;
            ps.setString(i++, "987654321");
            ps.setString(i++, "Lopez");
            ps.setInt(i++, 28002);
            ps.executeUpdate();

            System.out.println("Datos insertados correctamente.");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

- La sentencia INSERT tiene tres marcadores, uno para cada columna.
- Los valores se asignan en orden con setString() y setInt(). La variable i, que se incrementa con i++ en cada asignación, evita equivocarse al numerar los marcadores si más adelante se añaden columnas.
- Cada llamada a executeUpdate() envía los valores actuales e inserta una fila. Para el segundo cliente no se crea otra sentencia: basta con asignar valores nuevos.

## 5.4 Ejecución de sentencias DML de modificación de datos

Las sentencias INSERT, UPDATE y DELETE modifican el contenido de la base de datos. Se ejecutan con executeUpdate(), que devuelve un int con el **número de filas afectadas**. Ese número permite comprobar si la operación ha tenido efecto y cuántos registros han cambiado.

Este programa inserta cuatro clientes en la tabla CLIENTES con una sola sentencia:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class InsertarClientes {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        try (Connection c = DriverManager.getConnection(URL, USER, PASSWORD);
             Statement s = c.createStatement()) {

            int nFilas = s.executeUpdate(
                    "INSERT INTO CLIENTES (DNI, APELLIDOS, CP) VALUES "
                    + "('78901234X', 'NADALES', '44126'), "
                    + "('89012345E', 'HOJAS', NULL), "
                    + "('56789012B', 'SAMPER', '29730'), "
                    + "('09876543K', 'LAMIQUIZ', NULL)");
            System.out.println(nFilas + " filas insertadas.");

        } catch (SQLException e) {
            muestraErrorSQL(e);
        }
    }

    private static void muestraErrorSQL(SQLException e) {
        System.err.println("Mensaje: " + e.getMessage());
        System.err.println("SQLState: " + e.getSQLState());
        System.err.println("Codigo de error: " + e.getErrorCode());
    }
}
```

La salida es 4 filas insertadas.. El número de filas afectadas es especialmente útil con UPDATE y DELETE, porque nos dice si la condición del WHERE ha encontrado algo. Por ejemplo:

```
try (PreparedStatement ps = connection.prepareStatement(
        "UPDATE CLIENTES SET CP = ? WHERE DNI = ?")) {
    ps.setString(1, "30520");
    ps.setString(2, "89012345E");
    int filas = ps.executeUpdate();
    if (filas == 0) {
        System.out.println("No existe ningun cliente con ese DNI.");
    }
}
```

**Consejo:** para modificar datos que dependen de variables, usa siempre PreparedStatement, por los mismos motivos de seguridad que en las consultas.

# 6 Utilización del resultado de una consulta

Como hemos visto, el resultado de una consulta se guarda en un objeto ResultSet, que representa una tabla con las filas y columnas obtenidas. Hasta ahora lo hemos recorrido siempre de la misma forma: hacia delante, con next(), y solo para leer. En este apartado veremos que un ResultSet puede configurarse para moverse en ambas direcciones e incluso para modificar la base de datos.

## 6.1 Tipos de ResultSet y su configuración

Por defecto, un ResultSet **solo puede recorrerse hacia delante** y **no permite modificar** la base de datos. Para cambiar este comportamiento, al crear el Statement o el PreparedStatement se pasan dos parámetros adicionales, el **tipo de desplazamiento** y la **concurrencia**:

```
con.createStatement(tipo, concurrencia)
con.prepareStatement(sql, tipo, concurrencia)
```

**Tipo de desplazamiento**

- ResultSet.TYPE_FORWARD_ONLY (valor por defecto): el cursor solo puede avanzar.
- ResultSet.TYPE_SCROLL_INSENSITIVE: el cursor puede moverse en ambas direcciones. El ResultSet es una "foto" del momento de la consulta: no ve los cambios que otros usuarios hagan después en la base de datos.
- ResultSet.TYPE_SCROLL_SENSITIVE: se mueve en ambas direcciones y, en teoría, sí ve esos cambios.

**MySQL:** MySQL Connector/J no admite TYPE_SCROLL_SENSITIVE: si lo pedimos, lo trata como TYPE_SCROLL_INSENSITIVE.

Con los dos tipos SCROLL se pueden usar, además de next(), estos métodos de navegación:

| MÉTODO | MUEVE EL CURSOR A… |
|---|---|
| **previous()** | La fila anterior. |
| **first() / last()** | La primera / la última fila. |
| **absolute(int n)** | La fila número n (la primera es la 1). |
| **beforeFirst() / afterLast()** | La posición anterior a la primera fila / posterior a la última. |

Si el ResultSet es TYPE_FORWARD_ONLY, llamar a cualquiera de estos métodos lanza una SQLException.

**Concurrencia**

- ResultSet.CONCUR_READ_ONLY (valor por defecto): el ResultSet es de solo lectura.

- ResultSet.CONCUR_UPDATABLE: permite modificar la base de datos a través del ResultSet, con los métodos updateXXX() y updateRow() que veremos en el apartado 6.2.

Por ejemplo, este Statement genera resultados que se pueden recorrer en ambas direcciones y modificar:

```
Statement stmt = connection.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,   // bidireccional
    ResultSet.CONCUR_UPDATABLE           // actualizable
);
```

Y con un ResultSet desplazable podemos, por ejemplo, saber cuántas filas tiene y volver al principio para recorrerlo:

```
try (Statement st = connection.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                               ResultSet.CONCUR_READ_ONLY);
     ResultSet rs = st.executeQuery("SELECT nombre FROM usuarios")) {


    rs.last();                                          // ir a la ultima fila
    System.out.println("Usuarios: " + rs.getRow());     // numero de la fila actual
    rs.beforeFirst();                                   // volver al principio
    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }
}
```

## 6.2 Modificación de datos con ResultSet actualizable

Un ResultSet actualizable (CONCUR_UPDATABLE) permite modificar las filas obtenidas sin escribir una sentencia UPDATE. Es útil cuando el nuevo valor de cada fila se decide en Java, **fila a fila**: aplicando la lógica del programa, usando datos que no están en la base de datos o pidiendo confirmación al usuario. Por ejemplo, una pantalla que muestra los productos uno a uno y deja editar el precio de cada uno.

Para modificar una fila hay que seguir tres pasos:

1. **Situar el cursor** en la fila, con next(), absolute(n)…
2. **Cambiar el valor en el ResultSet** con el método updateXXX() del tipo adecuado (updateString(), updateInt(), updateBigDecimal()…). En este momento el cambio solo está en el ResultSet, no en la base de datos.
3. **Enviar el cambio a la base de datos** con updateRow().

Además de modificar, se puede **borrar** la fila actual con deleteRow() o **insertar** una nueva: primero se llama a moveToInsertRow(), que sitúa el cursor en una fila especial vacía; después se asignan los valores con updateXXX() y, por último, se llama a insertRow().

**MySQL:** en MySQL, para que el ResultSet sea actualizable, la consulta tiene que ser sobre **una sola tabla**, sin funciones (como SUM() o COUNT()) y debe **incluir la clave primaria**. Si no, al intentar modificar salta el error *Result Set not updatable*.

**Ejemplo: modificar los precios de una tabla**

Partimos de esta tabla de productos:

```sql
CREATE TABLE PRODUCTOS (
    id     INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    precio DECIMAL(8,2) NOT NULL
);
INSERT INTO PRODUCTOS VALUES
    (101, 'Teclado', 25.00), (102, 'Raton', 12.50), (103, 'Monitor', 149.90);
```

El método modificarPrecios() recorre la tabla y multiplica el precio de cada producto por un factor: 1.10 para subirlo un 10 % y 0.90 para bajarlo un 10 %. Como solo hay que recorrer la tabla una vez hacia delante, basta con un ResultSet TYPE_FORWARD_ONLY y CONCUR_UPDATABLE:

```java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ModificarPrecios {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASSWORD)) {
            modificarPrecios(con, new BigDecimal("1.10"));   // subir un 10 %
            System.out.println("Precios actualizados.");
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void modificarPrecios(Connection con, BigDecimal factor)
            throws SQLException {
        // Una sola tabla y con su clave primaria: requisito de MySQL
        String sql = "SELECT id, nombre, precio FROM PRODUCTOS";

        try (Statement stmt = con.createStatement(ResultSet.TYPE_FORWARD_ONLY,
                                                  ResultSet.CONCUR_UPDATABLE);
             ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                // 1. Leer el valor actual y calcular el nuevo (2 decimales)
                BigDecimal precio = rs.getBigDecimal("precio");
                BigDecimal nuevo = precio.multiply(factor)
                        .setScale(2, RoundingMode.HALF_UP);
                // 2. Cambiarlo en el ResultSet
                rs.updateBigDecimal("precio", nuevo);
                // 3. Enviarlo a la base de datos
                rs.updateRow();
            }
        } // try-with-resources cierra el ResultSet y el Statement
    }
}
```

Los precios se manejan con la clase BigDecimal y no con double. Los tipos float y double no pueden representar exactamente todos los decimales (por ejemplo, 0.1 + 0.2 da 0.30000000000000004), lo que provoca errores de redondeo en importes de dinero. BigDecimal trabaja con decimales exactos, igual que el tipo DECIMAL de SQL. El método setScale(2, RoundingMode.HALF_UP) redondea el resultado a dos decimales.

# 7 Ejecución de procedimientos almacenados en una base de datos

Los **procedimientos almacenados** son bloques de código SQL que pueden incluir variables, condiciones (IF), bucles y otras estructuras de control, como un pequeño programa. Se crean una sola vez en la base de datos, con CREATE PROCEDURE, y quedan guardados en ella. Se ejecutan directamente en el SGBD, así que una operación que requiere varias sentencias se resuelve en el servidor sin ir y volver a la aplicación en cada paso.

Desde Java **no se escribe el código del procedimiento**: solo se le llama por su nombre, pasándole los datos que necesita.

Sus características principales son:

- Pueden tener **parámetros de entrada** (IN), con los que reciben datos; **de salida** (OUT), con los que devuelven resultados; y **de entrada/salida** (INOUT), que hacen ambas cosas.
- Las **funciones almacenadas** (CREATE FUNCTION) son parecidas, pero además devuelven un valor, como un método de Java con return.
- La mayoría de los SGBD los admiten: MySQL, Oracle, SQL Server, PostgreSQL… Los gestores embebidos como SQLite no, así que esta parte de la unidad se trabaja solo con MySQL.

## 7.1 La interfaz CallableStatement

En JDBC, los procedimientos almacenados se ejecutan con la interfaz CallableStatement, que funciona de forma muy parecida a PreparedStatement:

1. **Creación.** Se obtiene con el método prepareCall() de Connection, usando la sintaxis estándar de JDBC "{call Nombre(?, ?)}", con un ? por cada parámetro. El driver traduce esta sintaxis a la sentencia CALL de cada gestor, así que es la misma para todos.
2. **Asignación de parámetros.** Los parámetros de entrada se rellenan con los métodos setXXX(), igual que en una sentencia preparada.
3. **Ejecución**, con el método que corresponda a lo que hace el procedimiento:
   - Si devuelve filas (contiene un SELECT): executeQuery(), que devuelve un ResultSet.
   - Si modifica datos: executeUpdate(), que devuelve el número de filas afectadas.
   - Si tiene parámetros de salida, o no sabemos qué devuelve: execute().

**Ejemplo 1: procedimiento que devuelve filas**

Trabajaremos con esta tabla de álbumes musicales:

```
CREATE TABLE album (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    titulo  VARCHAR(100) NOT NULL,
    artista VARCHAR(100) NOT NULL
);
INSERT INTO album (titulo, artista) VALUES
    ('Abbey Road', 'The Beatles'), ('Thriller', 'Michael Jackson');
```

El procedimiento DameAlbumes recibe un título y un artista, y devuelve los álbumes que coinciden con ambos:

```
DELIMITER //
CREATE PROCEDURE DameAlbumes(IN p_titulo VARCHAR(100), IN p_artista VARCHAR(100))
BEGIN
    SELECT id, titulo, artista FROM album
    WHERE titulo = p_titulo AND artista = p_artista;
END //
DELIMITER ;
```

La orden DELIMITER es necesaria al crear el procedimiento desde phpMyAdmin o desde la consola de MySQL. El cuerpo del procedimiento contiene sentencias terminadas en ;, y sin ella MySQL interpretaría el primer ; como el final del CREATE PROCEDURE. Por eso se cambia temporalmente el delimitador a // y, al terminar, se recupera el ;.

Como el procedimiento contiene un SELECT, se ejecuta con executeQuery() y el resultado se recorre como el de cualquier consulta:

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;

public class LlamarDameAlbumes {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             CallableStatement cs =
                     connection.prepareCall("{call DameAlbumes(?, ?)}")) {

            // Asignar valores a los parametros de entrada
            cs.setString(1, "Thriller");
            cs.setString(2, "Michael Jackson");

            // Contiene un SELECT: executeQuery() devuelve un ResultSet
            try (ResultSet rs = cs.executeQuery()) {
                while (rs.next()) {
                    System.out.println("Album: " + rs.getString("titulo"));
                }
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

**Ejemplo 2: procedimiento que modifica datos**

La llamada desde JDBC es igual en todos los gestores, pero la definición del procedimiento usa la sintaxis propia de cada uno. Esta es la de MySQL. El procedimiento ActualizarPrecio ajusta el precio de un producto de la tabla PRODUCTOS (apartado 6.2)

multiplicándolo por un porcentaje. Recibe dos parámetros: el identificador del producto y el porcentaje de ajuste.

```
DELIMITER //
CREATE PROCEDURE ActualizarPrecio(IN productId INT, IN porcentaje FLOAT)
BEGIN
    UPDATE PRODUCTOS SET precio = precio * porcentaje WHERE id = productId;
END //
DELIMITER ;
```

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class EjecutarProcedimiento {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             CallableStatement cs =
                     connection.prepareCall("{call ActualizarPrecio(?, ?)}")) {

            cs.setInt(1, 101);       // ID del producto
            cs.setFloat(2, 1.10f);   // subida del 10 %

            int filasAfectadas = cs.executeUpdate();
            System.out.println("Filas afectadas: " + filasAfectadas);

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

1. **Preparación:** prepareCall("{call ActualizarPrecio(?, ?)}") prepara la llamada con dos marcadores, uno por parámetro.
2. **Asignación:** setInt(1, 101) indica el producto y setFloat(2, 1.10f) el porcentaje de ajuste.
3. **Ejecución:** como el procedimiento modifica datos, se usa executeUpdate(), que devuelve el número de filas afectadas.

### Ejemplo 3: procedimiento con un parámetro de salida

Cuando un procedimiento devuelve un dato a través de un parámetro OUT, hay que indicarlo antes de ejecutarlo con registerOutParameter(), junto con su tipo SQL. Después de ejecutarlo con execute(), el valor se lee con el método getXXX() correspondiente. Por ejemplo, para este procedimiento, que cuenta los álbumes de la tabla:

```
DELIMITER //
CREATE PROCEDURE ContarAlbumes(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM album;
END //
DELIMITER ;
```

la llamada desde Java es:

```
try (CallableStatement cs = connection.prepareCall("{call ContarAlbumes(?)}")) {
    cs.registerOutParameter(1, Types.INTEGER);   // el ? 1 es de salida
    cs.execute();
    System.out.println("Numero de albumes: " + cs.getInt(1));
}
```

## Ventajas e inconvenientes de los procedimientos almacenados

| Ventajas | Inconvenientes |
|---|---|
| **Rendimiento:** el trabajo se hace en el servidor, lo que reduce el tráfico de datos entre la aplicación y la base de datos. | **Dependen del gestor:** cada uno tiene su propia sintaxis. Si se cambia de MySQL a Oracle, hay que reescribirlos, y SQLite ni siquiera los admite. |
| **Reutilización:** un mismo procedimiento puede usarse desde varias aplicaciones. | **La lógica queda repartida** entre Java y la base de datos, lo que hace más difícil depurarla, probarla y controlar sus versiones con Git. |
| **Seguridad:** se puede permitir a un usuario ejecutar un procedimiento sin darle acceso directo a las tablas. | |

# 8 Gestión de transacciones

Una **transacción** es un conjunto de operaciones SQL que se ejecutan como **una sola unidad de trabajo**: o se completan todas, o no se aplica ninguna. Así se garantiza que los datos sigan siendo coherentes aunque algo falle a mitad del proceso.

El ejemplo clásico es una transferencia bancaria: hay que restar el dinero de la cuenta de origen y sumarlo en la de destino. Si el programa falla después de restar y antes de sumar, el dinero desaparecería. Agrupando las dos operaciones en una transacción, si la segunda falla, se deshace también la primera.

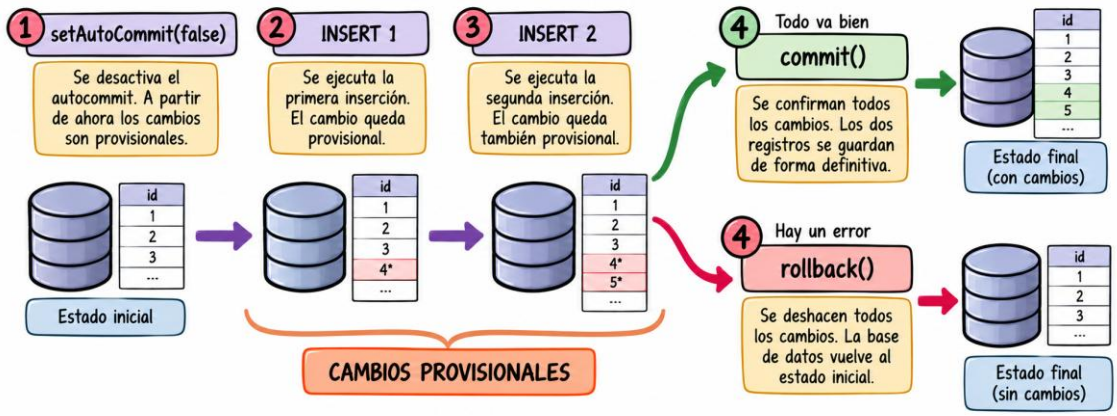
### Características ACID de las transacciones

Las transacciones cumplen cuatro propiedades, conocidas por sus iniciales en inglés como **ACID**:

1. **Atomicidad** (*Atomicity*): se completan todas las operaciones de la transacción o no se realiza ninguna. Si alguna falla, se cancelan todas.
2. **Consistencia** (*Consistency*): la transacción lleva la base de datos de un estado válido a otro, respetando todas las reglas de integridad (claves primarias, claves ajenas, NOT NULL…).
3. **Aislamiento** (*Isolation*): una transacción se ejecuta sin interferir con las demás transacciones que estén en curso a la vez.
4. **Durabilidad** (*Durability*): una vez confirmada, la transacción es permanente, aunque después se produzca un fallo del sistema.

### Implementación de transacciones en JDBC

En JDBC, las transacciones se controlan desde el objeto Connection mediante el modo **auto-commit**. Por defecto está activado: **cada sentencia se confirma automáticamente** en cuanto se ejecuta, como si fuera una transacción en sí misma. Para agrupar varias sentencias en una sola transacción, hay que **desactivar el auto-commit** y decidir nosotros cuándo se confirman los cambios.




### Pasos para gestionar una transacción en JDBC

1. **Desactivar el auto-commit** con setAutoCommit(false). A partir de ese momento, las sentencias que se ejecuten no se confirman automáticamente.

2. **Ejecutar las operaciones** que forman la transacción (por ejemplo, varios INSERT o UPDATE).
3. **Confirmar o deshacer los cambios:**
   - si todas las operaciones se han ejecutado correctamente, se llama a commit() para confirmarlas;
   - si se produce un error, se llama a rollback() para deshacer todos los cambios hechos desde el inicio de la transacción.

### Ejemplo de transacción en JDBC

Este programa inserta dos álbumes en la tabla album (apartado 7.1) dentro de una transacción. Si las dos inserciones van bien, se confirman con commit(). Si alguna falla, se deshacen con rollback().

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class GestionTransacciones {

    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD)) {
            // 1. Desactivar auto-commit: empieza la transaccion
            connection.setAutoCommit(false);

            try (Statement statement = connection.createStatement()) {
                // 2. Operaciones que forman la transaccion
                statement.executeUpdate("INSERT INTO album (titulo, artista) "
                        + "VALUES ('Album1', 'Artista1')");
                statement.executeUpdate("INSERT INTO album (titulo, artista) "
                        + "VALUES ('Album2', 'Artista2')");

                // 3a. Todo ha ido bien: confirmar
                connection.commit();
                System.out.println("Transaccion completada con exito.");
            } catch (SQLException e) {
                // 3b. Algo ha fallado: deshacer todo
                System.err.println("Error en la transaccion. Se deshacen cambios.");
                connection.rollback();
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

1. **Desactivar el auto-commit:** con setAutoCommit(false), el primer INSERT no se confirma hasta que sepamos si el segundo también funciona.
2. **Ejecutar las operaciones:** las dos sentencias INSERT se ejecutan dentro de un try interior.
3. **Confirmar o revertir:** si se llega al final del try sin errores, commit() guarda las dos filas. Si cualquiera de los INSERT lanza una SQLException, el programa salta al catch y rollback() deshace también la que se hubiera insertado.

**Prueba:** para comprobar el rollback(), cambia el segundo INSERT para que falle (por ejemplo, escribe mal el nombre de una columna) y ejecuta el programa: verás que en la tabla tampoco aparece el primer álbum.

### Importancia de las transacciones

- **Garantía de integridad:** aseguran que las operaciones formadas por varios pasos se completen enteras, sin dejar los datos a medias.
- **Recuperación ante fallos:** si algo falla, rollback() deshace los cambios y la base de datos vuelve a un estado coherente.
- **Desarrollo seguro:** la forma de gestionar transacciones en JDBC es la misma con cualquier gestor, lo que facilita desarrollar aplicaciones fiables.

# 9 Bibliografía

[1] Cortijo Bon, Carlos A. *Acceso a datos*. Editorial Síntesis (España). ISBN: 978-84-9171-356-2.

[2] Córcoles Tendero, José Eduardo. *Acceso a datos*. Editorial RA-MA (España). E-ISBN: 9788499643908.

[3] Camarena Estruch, Joan Gerard; Múrcia Andrés, José A. *Acceso a datos*. Editorial McGraw Hill. ISBN: 9788448626730.